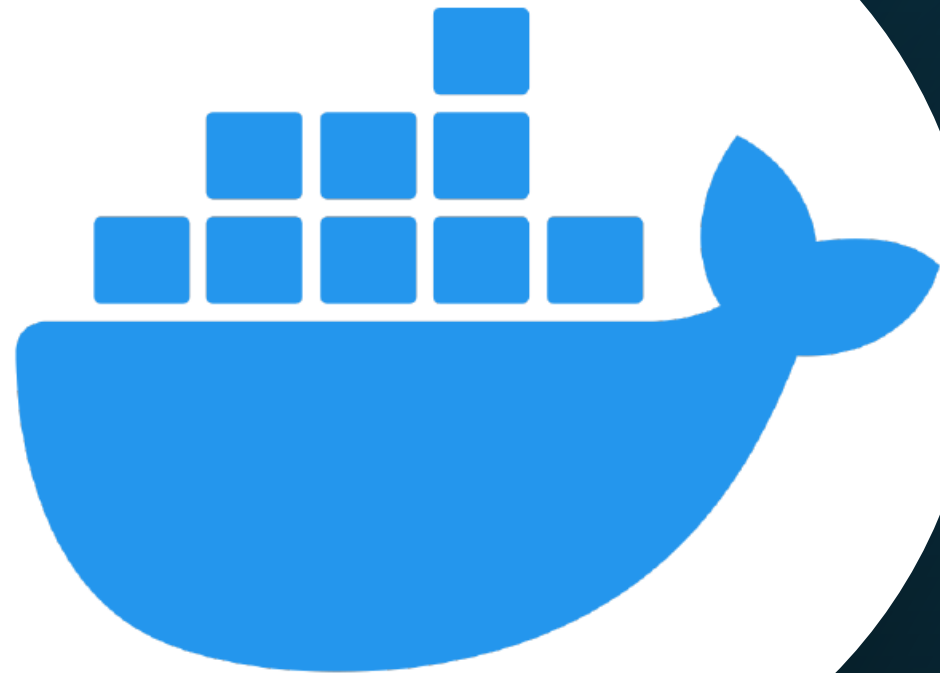


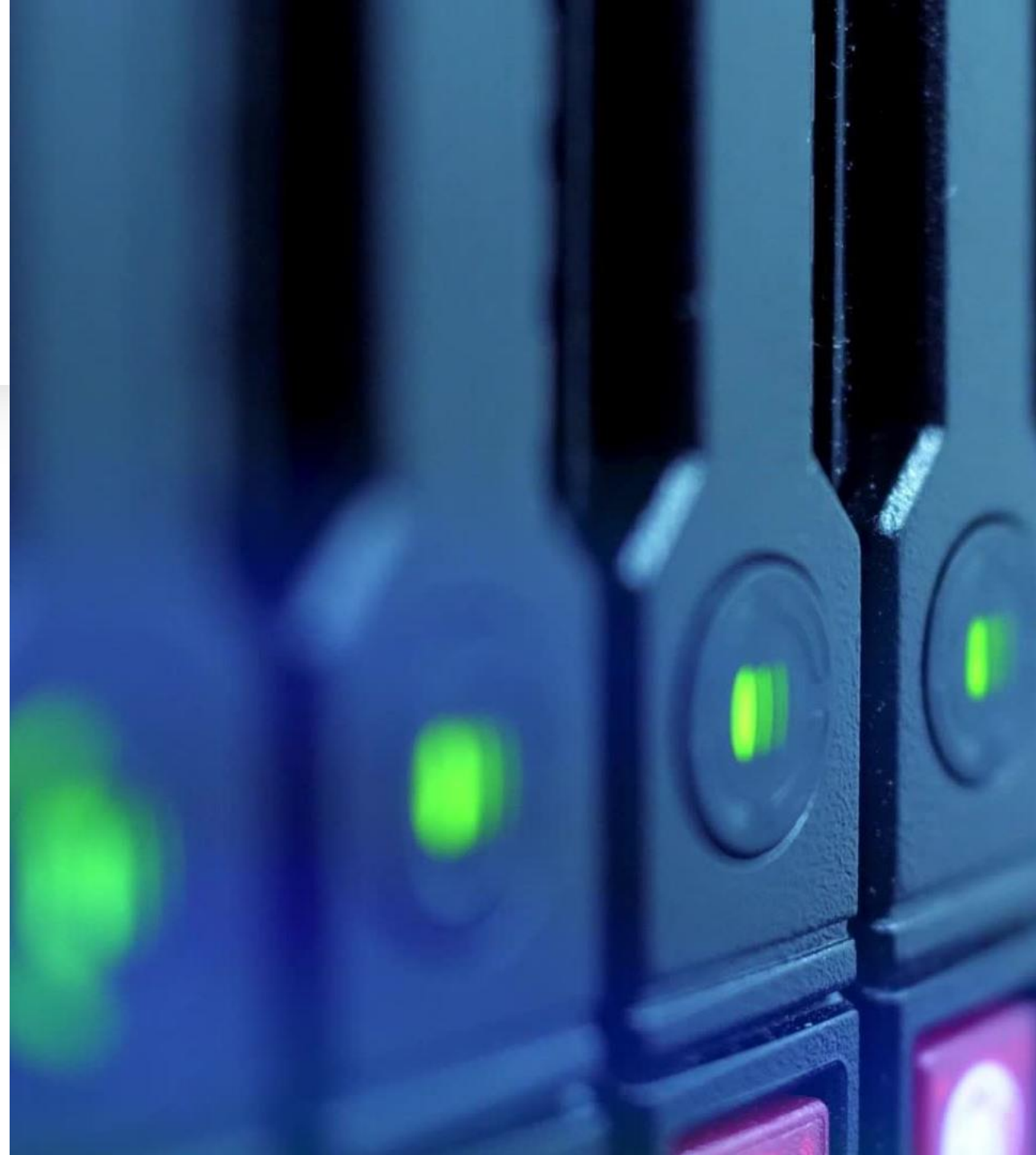
What is and how to use docker

Jarrett Lane



Why docker is important

- Lets you run things, no matter what type of system you have or what you have installed
- Lets you run things isolated from everything else
- Quickly and easily run things
- Great for servers, cybersecurity labs, development, and so much more
- Widely adopted by many organizations



Virtualization and containerization

-
- Before explaining docker it's important to understand virtualization and containerization
 - Understanding this will make learning docker so much easier
 - Remember, everything on your computer is all just electricity, how do we make all of these bits act like a system?

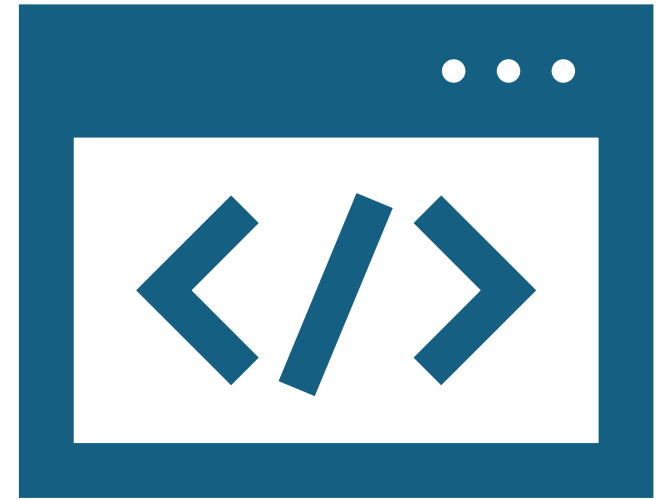


An analogy that helped me understand

- Let's say we have a huge grass field, we can use this field for anything we want
- Now let's say we have local football and soccer teams, they don't play the same sport, but that's ok because they can split the field and do their own thing
- But what if there are two football teams, high school and little league, why not let them practice together, they play the same sport right?
- How about instead we separate the football field for the separate teams, that's much safer

How the analogy applies

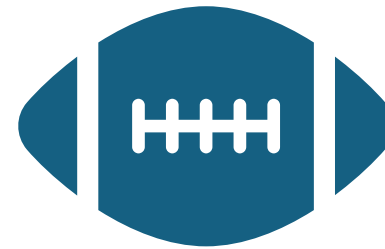
- You have a ton of hardware, you can do whatever with it
- You have two operating systems that use a completely different architecture, ok then share the hardware (Virtualize)
- You have an app that needs to run separate from everything else on the operating system, ok, create an isolated portion of the operating system for that app (Containerize)



Note



The definition of virtualization in computer science is basically to play pretend, this is very useful for many reasons



We pretended the big grass fields were soccer and football fields, that doesn't change the fact it's a grass field

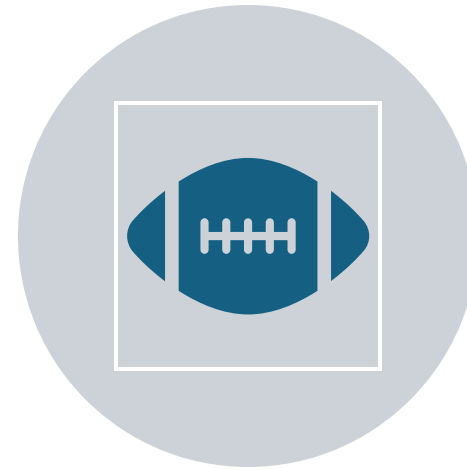
Docker

-
- One of the most popular containerization options
 - Open source
 - A flexible and easy way to handle containers
 - Using docker is made up of 3 simple parts
 - Docker file
 - Image
 - Container
 - I will explain each of these and tie them back to the analogy

Docker file



BASICALLY A FILE TELLING DOCKER "I NEED THESE THINGS SET UP IN THIS WAY"



IMAGINE A FOOTBALL TEAM TELLING THE FIELD STAFF
"WE NEED THIS EQUIPMENT AND WE NEED IT SET UP
IN THIS WAY FOR OUR PRACTICE TONIGHT"

Docker Image

-
- Remember the stuff the docker file asked for?
 - That's what the image is, the stuff that was asked for
 - It's a template, wtf is a template?
 - You may have multiple practices, and they may not all be the same, but they will always have the same equipment set up the same way
 - So a template is the basic setup for something that can be built upon in different ways

Docker container

-
- A container is an instance of an image, wtf do I mean "instance"!!!
 - Back to the field, so now you have the necessary equipment and setup, but who is going to use it? The team will
 - Who is going to use the image? the running container will
 - An instance is both the container running code and the football team practicing
 - So instance means "Something is happening from this"

Docker making this all easier

- Docker brings all of these things together
- It knows how to read the list, put together the image and set it up properly, and run a process from that image
- This makes running things so much easier, instead of having to manually install certain dependencies and set up the app, you just give docker the dockerfile and it knows exactly what to do and sets everything up
- Since docker runs everything isolated, you won't get a ton of weird errors because the program has everything it needs and nothing else is messing with it

Using docker



Now that makes so much more sense right?



Using docker is also pretty straight forward



Here is what you need to do:

- Install it (If you haven't)
- Make docker file
- Use that file to make an image
- Run a container from that image

Install docker

- You can download it from the website, select the one for your OS
- You can also use a package manager and download it with command line
- You can use the GUI or command line, I like the GUI better because g-d gave me eyes for a reason (You will need to run a few commands)
- If you are on linux you probably already have docker installed



The example I will use

- I will use a basic python flask server that returns a basic json message when accessed
- Here is the code:
 - `import flask`
 - `from flask import jsonify`
 - `app = flask.Flask(__name__)`
 - `@app.route("/")`
 - `def index():`
 - `return jsonify(message = "Working docker example")`
 - `if __name__ == "__main__":`
 - `app.run(host="0.0.0.0", port=5000)`

Quick note

A docker image is made up of layers

In our example, python is the parent/base layer since it is the basic foundational part of the image necessary, then off of python we can use flask

Sometimes the parent layer may be a lightweight linux image

Higher layers depend on their parent/lower layers

You can build upon the layers depending on what you need



Make the docker file

- Docker files are made up of commands
- Here are some basic ones:
 - FROM: Specify the parent image/layer, you will build upon this
 - WORKDIR: specify where docker should run necessary commands when running the image, where do you want the container to run?
 - RUN: run commands necessary for the container, like installing the packages you need
 - COPY: copy files from directories
 - ADD: like COPY, but can also use URLs and Directories, and also handles compressed files
 - ENTRYPOINT: the first command that gets run when you start a container
 - CMD: give a command to run when starting a container, very similar to ENTRYPOINT but it can be overridden, ENTRYPOINT can't
 - EXPOSE: which port your container will communicate on

Make the docker file

- Here is how we are making out docker file for the flask server:
 - FROM python:3.10
 - "I need python 3.10 as a basis for running this server"
 - ADD example.py .
 - "Here is the code that the container will run, put it in the container's current directory"
 - RUN pip install flask
 - "I need flask so install that package for me, good boyyy"
 - CMD ["python", "./example.py"]
 - "Run this command, which starts the python code that I want to run."

Making the image

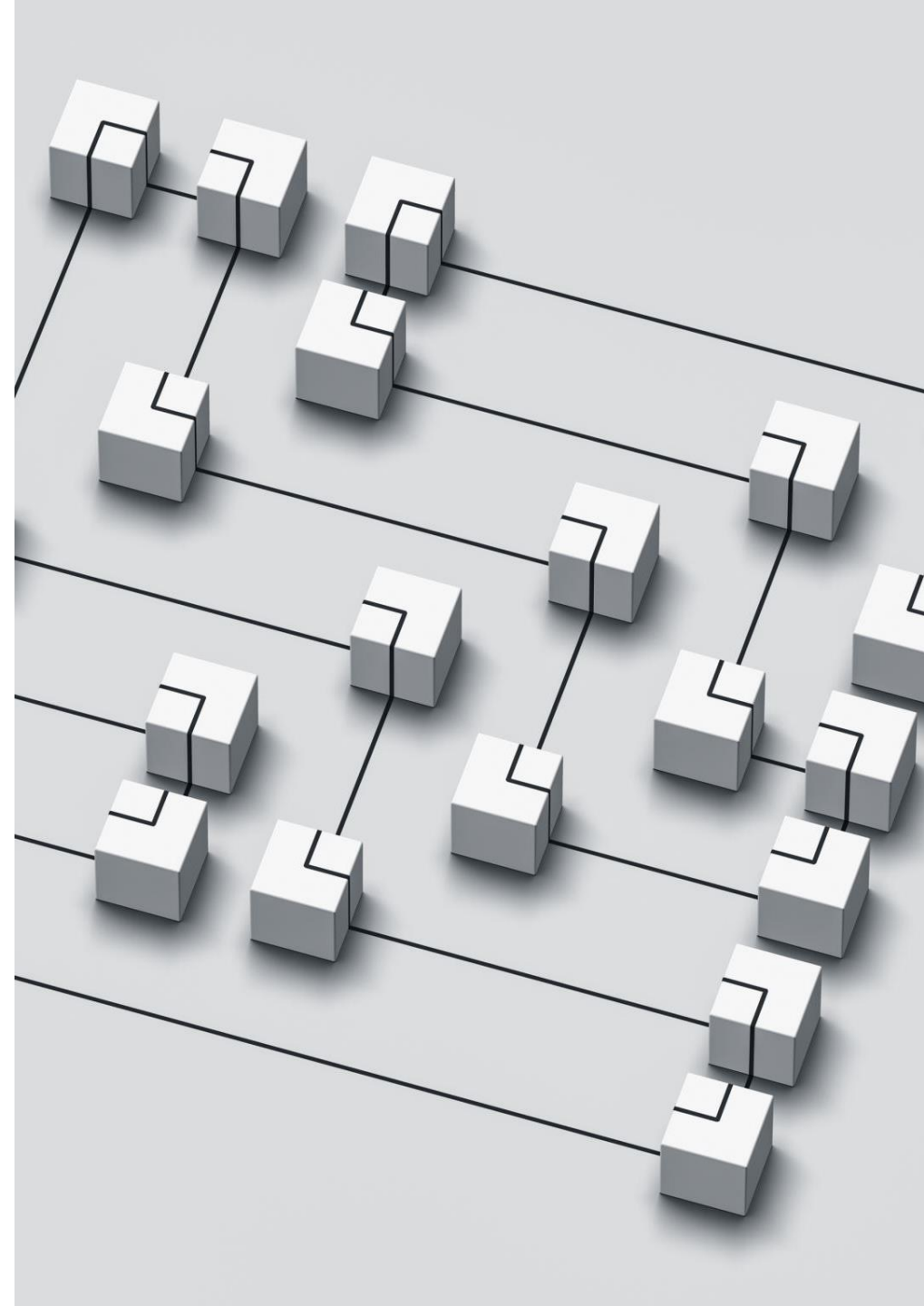
- Now from the file you make the image
- Go to command line and make sure you are in the same directory as your docker file
- Run: `docker build -t example-image .`
- This command is basically saying "Make the image from the dockerfile in this directory and name it example-image"
- Some images can get pretty big, if space is an issue for you, you can try optimizing them or just deleting them when you are done

Running the container

- Run this command: `docker run -d -p 8082:5000 example-image`
- This is saying "Start a container from example-image, chill with the logs, and publish the port 8082:5000"
- What is publishing a port?
- When you run something Isolated, EVERYTHING is isolated (initially), including the network of the container
- We can't access port 5000 directly, so we tell the container: "Your port 5000 should talk to us through our local port 8082"
- Go to your browser and try <http://localhost:8082>
- Stop the container when finished

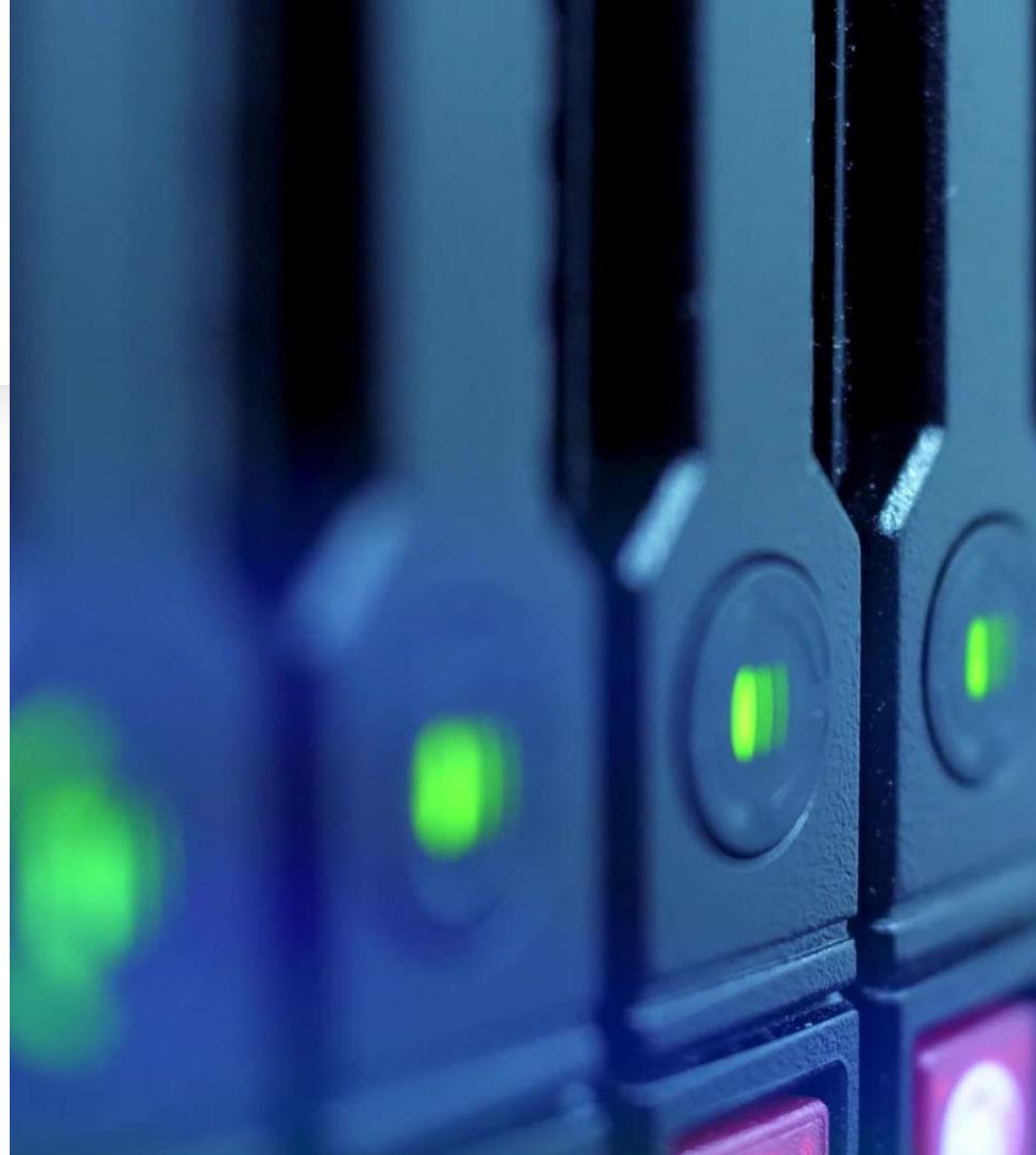
Note on publishing ports

- There are other ways to communicate with the docker instance and your host machine
- Docker is naturally isolated
- Important Distinction: EXPOSE vs. PUBLISH (-p or -P)
 - EXPOSE:
 - Documents the ports the container listens on. It does not make the port accessible from the host machine or external networks.
 - PUBLISH (-p or -P):
 - Maps a container port to a host port, making it accessible from the host machine and potentially external networks.
- While writing these slides I found this so be careful:
 - <https://bgray-dev.medium.com/docker-ports-what-are-you-really-publishing-df473669093c>



Virtual networking

- A network is a collection of connected devices
- A virtualized/containerized system thinks it is its own device
- When you have multiple virtual devices on your system that think they are devices, and they are connected, you have a network running on your computer!!!
- By default, containers don't recognize the host OS that is connected to the network, they just think they are isolated on their own "network"



Important Notes

- This is a VERY basic introduction to docker but here are a few other good things to know:
- Docker compose
 - Docker compose is an efficient way to deal with multiple containers and easily reproduce desired containers. It is basically a config file in YAML that is easier to read, type out, and use with containers. This is so much nicer than typing out a command with a lot of parameters.
- Volumes
 - Basically, containers sometimes need data or they generate data. We may need data from the container too, the problem is that restarting or deleting a container can cause all the data associated with that container to be lost. To fix this we use volumes, now we can access and save data that is generated by or needed by the container, even if the container is reset or deleted.
- Docker hub
 - What if there is an image that has already been generated for what you need, this is where docker hub comes in. Think about it like a GitHub but for docker images. This offers countless benefits like efficiency, automation, security, access to many images, and collaboration. You can also upload your own image if you want.
- Docker security
 - If you plan on using docker more that is great, just remember to keep security in mind. For the most part Docker is very secure due to its isolation, but understanding the associated vulnerabilities of docker is important to know, especially if you want to use Docker to host servers on the internet.

Want to play with it more? (Pause)

-
- I made these two fun ctf style challenges a while ago:
 - https://github.com/JarrLane/Offensive_Meeting_Content/tree/main/Containers_explained/challenge_part_1
 - https://github.com/JarrLane/Offensive_Meeting_Content/tree/main/Containers_explained/challenge_part_2
 - Also experiment with it a little bit too

References

- Myself: https://github.com/JarrLane/Offensive_Meeting_Content/blob/main/Containers_explained/containers.md
- I wrote this over the summer, this PowerPoint is a summary of what I wrote

